

Componentes

1. Componentes

Último tema del módulo. Todo lo que has construido hasta ahora — repositorios, servicios, controladores — lo vas a mirar ahora bajo una óptica distinta: la de componente. Vas a identificar un problema que ya tienes en tu propio código desde el Tema 3, y resolverlo construyendo un componente con un contrato explícito — primero en una dirección, con PostgreSQL (Actividad 4.1), y después exactamente con el mismo molde en la dirección contraria, con MongoDB (Actividad 4.2).

Qué es la programación orientada a componentes

Piensa en el cargador de tu móvil. Lo enchufas a cualquier teléfono con el mismo conector y funciona: no necesitas saber qué circuitos tiene dentro, ni el teléfono necesita saber qué marca de cargador es. Si se estropea, compras otro con el mismo conector y todo sigue igual, sin tocar el teléfono para nada. Lo único que importa es el conector — la forma, el voltaje que da—; cómo convierte la electricidad por dentro es indiferente para quien lo usa.

Idea clave — Un componente de software es la misma idea aplicada a código: una pieza autocontenida, con una responsabilidad concreta, que expone un contrato público — lo único que hace falta conocer para usarla — sin que quien la usa necesite saber cómo resuelve ese trabajo por dentro.

Ya usas componentes, sin haberlos llamado así

Tus servicios gestionados por Spring y los repositorios de Spring Data son ejemplos de piezas que otras clases pueden utilizar sin conocer cómo resuelven internamente su trabajo. Eso es, en esencia, la programación orientada a componentes: ensamblar la aplicación con piezas autocontenidas y reutilizables, cada una con un contrato público — nunca cómo resuelve esa responsabilidad por dentro. Ese contrato no siempre es una interfaz Java separada:

Pieza	¿Interfaz separada?	Cuál es el contrato
@Service normal (LibroService)	No	Sus métodos públicos; los consumidores dependen de la clase concreta
@Repository (LibroRepository)	Sí	La interfaz LibroRepository, que extiende JpaRepository; Spring genera su implementación
El patrón que construyes hoy	Sí, a propósito	Una interfaz aparte, con la implementación oculta detrás

El vocabulario, heredado de los JavaBeans

Ese vocabulario viene de una época anterior a Spring — la de los JavaBeans y los entornos visuales, donde se arrastraban piezas ya hechas sobre un lienzo. No hace falta haber visto nada parecido, pero el vocabulario sigue vivo hoy, con su equivalente en Spring Boot:

Concepto (JavaBeans)	Qué significa	Equivalente en Spring Boot
Propiedades	Valores configurables sin tocar el código	Campos de un bean — p. ej. <code>corePoolSize</code> de un <code>TaskExecutor</code>
Eventos	Acciones que el componente dispara, sin conocer a quién	<code>ApplicationEventPublisher</code> + <code>@TransactionalEventListener</code>
Persistencia/serialización	Guardar y recuperar el propio estado	Poco relevante: tus <code>@Service</code> no suelen guardar estado propio
Empaquetado	Distribuir sin ver el código fuente	Una dependencia en tu <code>pom.xml</code>

Quién ensambla las piezas

Lo único que cambia de verdad es quién ensambla las piezas: antes lo hacía el propio IDE, al arrastrarlas sobre el lienzo; ahora lo hace Spring, detectando tus componentes (`@Service`, `@Repository`, `@Component...`) e inyectando sus dependencias — normalmente por constructor, como llevas haciendo con `@RequiredArgsConstructor`.

Ventajas e inconvenientes

Como cualquier decisión de diseño, programar por componentes trae ventajas claras, pero no es gratis:

Ventajas	Inconvenientes
Reutilización — la misma pieza sirve en varios sitios	Más interfaces que mantener
Sustituibilidad — cambias la implementación sin tocar quien la usa	Más indirección — seguir el flujo cuesta un poco más
Pruebas aisladas — pruebas cada pieza por separado	
División del trabajo — cada componente, una responsabilidad	

Indirección: para ver qué hace de verdad un método, ya no basta con mirar una clase — primero encuentras la interfaz, y luego buscas su implementación real. Es el precio de la sustituibilidad.

Herramientas de desarrollo de componentes

Aquí no aplica una herramienta visual de arrastrar y soltar — no tiene sentido en un proyecto backend, sin interfaz gráfica. La "herramienta" en este contexto es Spring en sí mismo: vía `@Service`, `@Repository`, `@Component` y la inyección con `@RequiredArgsConstructor`, es Spring quien gestiona el ciclo de vida y el ensamblado de los componentes de tu aplicación.

Nivel 1: un repositorio, ya es un componente

```
public interface LibroRepository extends JpaRepository<Libro, Long> {  
}
```

En su forma más básica, LibroRepository ya es un "componente con conector a base de datos": una interfaz que Spring implementa y gestiona por ti — se declara una vez y se reutiliza donde haga falta, sin que nadie que lo use necesite saber cómo genera Spring esa implementación por debajo.

Nivel 2: cuando un repositorio no basta

Recuerda `NotaLecturaService`, del Tema 3: para comprobar que un libro existe antes de tocar Mongo, inyecta `LibroRepository` directamente. El problema no es "inyectar un repositorio" en general — `NotaLecturaService` también inyecta el suyo propio, y eso no tiene nada de malo. El problema es cruzar la frontera hacia el repositorio de otro módulo: `LibroRepository` pertenece al catálogo, no a notas de lectura.

```
// NotaLecturaService, tal y como lo dejaste en el Tema 3
private final LibroRepository libroRepository;

public List<NotaLecturaResponseDTO> findByLibroId(Long libroId) {
    if (!libroRepository.existsById(libroId)) {
        throw new ResponseStatusException(HttpStatus.NOT_FOUND, ...);
    }
    // ...
}
```

Un cambio concreto: el borrado lógico

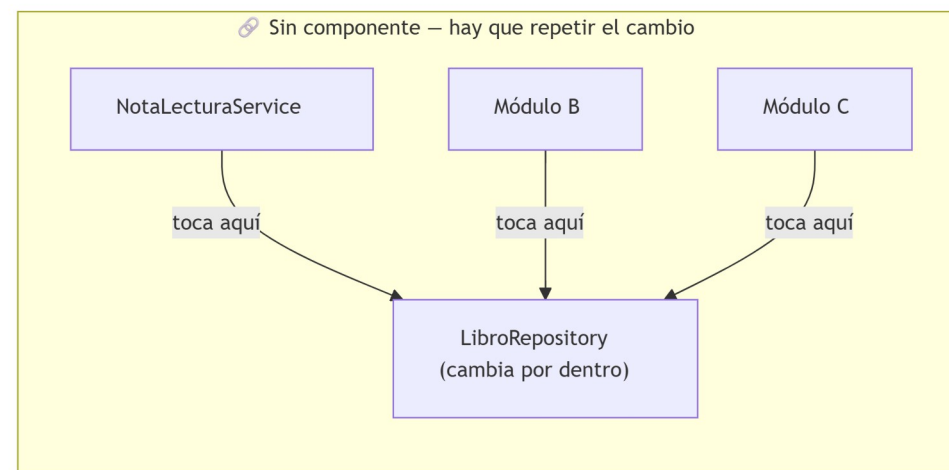
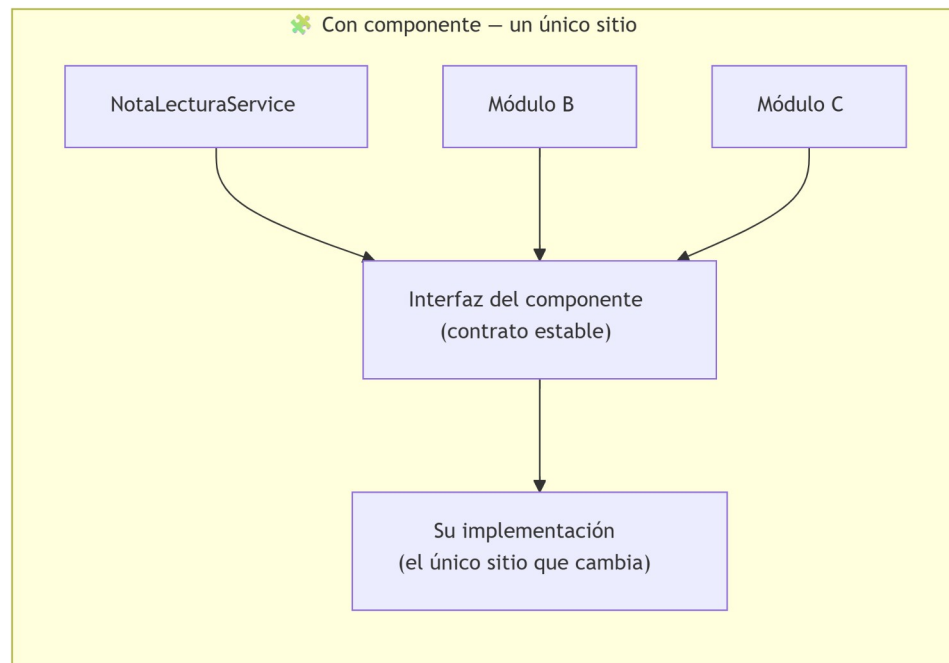
Funciona, pero acopla el módulo de notas de lectura a un detalle muy interno del catálogo: su repositorio JPA concreto. Imagina que mañana el catálogo decide dejar de borrar libros de verdad, y pasa a marcarlos como activo = false (borrado lógico, para poder recuperarlos si alguien se equivoca):

	Antes	Después del borrado lógico
Cómo se comprueba "el libro existe"	<code>libroRepository.existsById(libroId)</code>	<code>libroRepository.existsByIdAndActivoTrue(libroId)</code>

Esa decisión es solo cosa del catálogo. Pero si `NotaLecturaService` inyecta `LibroRepository` directamente, alguien tiene que acordarse de ir también a ese módulo y cambiar la llamada ahí.

Multiplicado por cada módulo

Multiplica ese problema por cada módulo que inyecte LibroRepository directamente, y un cambio interno del catálogo se convierte en un cambio que hay que repetir por toda la aplicación:



En "Sin componente", cada módulo apunta a un detalle interno que puede cambiar — tres sitios que tocar. En "Con componente", todos dependen de una interfaz estable; mientras el contrato no cambie, los consumidores no se enteran de nada.

Construyendo el componente: interfaz + implementación oculta

La solución es una interfaz pequeña, pensada para que otros módulos la usen, con una implementación que sí conoce los detalles pero permanece oculta. Dos modificadores de Java hacen todo el trabajo:

Modificador	¿Quién puede ver y usar la clase?
public	Cualquier clase, de cualquier paquete, con solo importarla
(sin modificador — package-private)	Solo las clases que viven en su mismo paquete

El contrato, en un paquete api — pensado para lo que otros módulos SÍ pueden ver — lleva public, a propósito: quieres que cualquier módulo lo importe y lo use.

El contrato y su implementación oculta

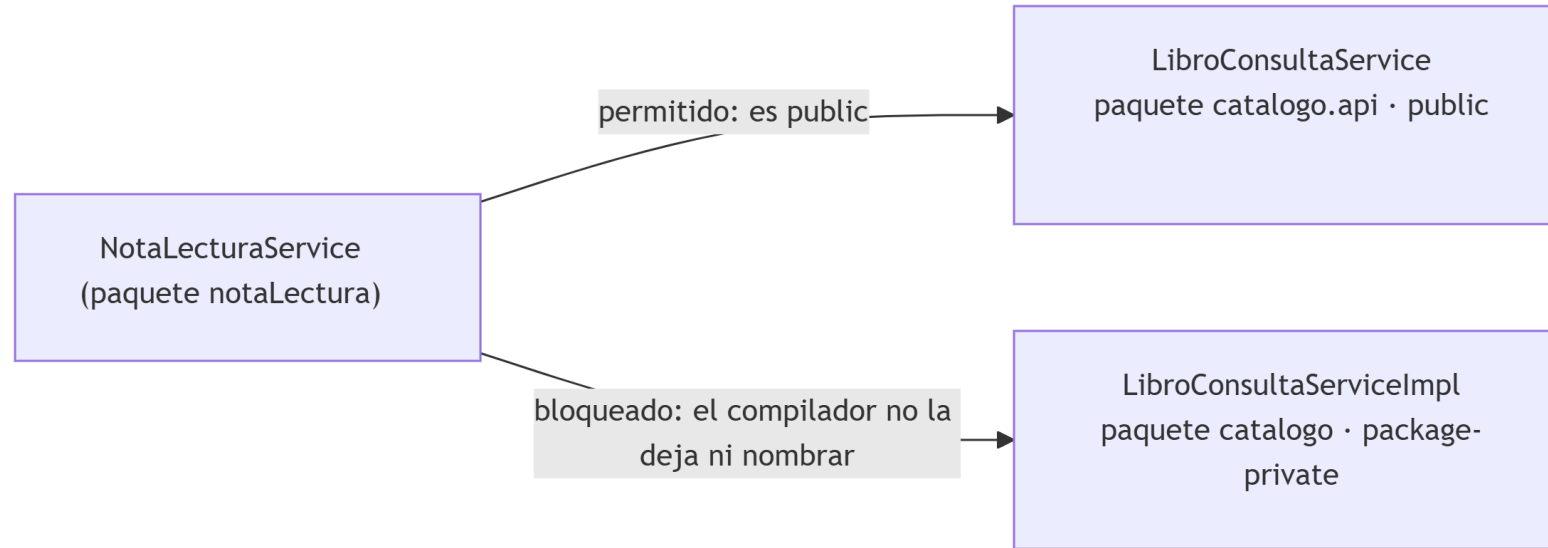
catalogo.api · public

```
public interface LibroConsultaService {  
    boolean existeLibro(Long libroId);  
}
```

catalogo · package-private (sin "public")

```
@Service  
@RequiredArgsConstructor  
class LibroConsultaServiceImpl implements LibroConsultaService {  
    private final LibroRepository libroRepository;  
    @Override  
    public boolean existeLibro(Long libroId) {  
        return libroRepository.existsById(libroId);  
    }  
}
```

Así queda repartida la visibilidad



`NotaLecturaService` puede importar y usar `LibroConsultaService` sin ningún problema — es `public`, para eso está. Pero no tiene ninguna forma de escribir `LibroConsultaServiceImpl` en su código: el compilador ni se lo permite, porque esa clase no existe para nadie fuera de `catalogo`.

La implementación no forma parte del API público

⚠ Aunque estuviera dentro de "api"...

Aunque LibroConsultaServicImpl estuviera en catalogo.api y siguiera siendo package-private, seguiría siendo invisible para las clases de otros paquetes: en Java, package-private solo permite acceso desde el mismo paquete. Aun así, conviene mantenerla fuera de api, en catalogo, para que la estructura deje clara la intención: api contiene el contrato público; catalogo, los detalles internos.

NotaLecturaService, ahora

NotaLecturaService pasa a depender solo de la interfaz, nunca de LibroRepository ni de la clase que la implementa:

```
// NotaLecturaService, ahora
private final LibroConsultaService libroConsultaService;

public List<NotaLecturaResponseDTO> findByLibroId(Long libroId) {
    if (!libroConsultaService.existeLibro(libroId)) {
        throw new ResponseStatusException(HttpStatus.NOT_FOUND, ...);
    }
    // ...
}
```

Probar el componente en aislamiento

LibroConsultaServiceImpl depende de una única pieza, LibroRepository, inyectada por constructor, así que puedes probar su lógica sin arrancar Spring y sin ninguna base de datos real — basta con sustituir esa dependencia por un doble de prueba con Mockito, el mismo principio que ya conoces de `@MockitoBean` (Tema 1), solo que aquí ni siquiera hace falta levantar el contexto de Spring:

- `@ExtendWith(MockitoExtension.class)`: activa Mockito para esta clase de test, sin contenedor de Spring — mucho más rápido que un `@SpringBootTest`.
- `@Mock`: crea el doble de prueba de LibroRepository — decides tú, línea a línea, cómo responde.
- `@InjectMocks`: crea la instancia real de LibroConsultaServiceImpl e inyecta los `@Mock` por constructor.

El test, con Mockito

```
@ExtendWith(MockitoExtension.class)
class LibroConsultaServiceImplTest {

    @Mock
    private LibroRepository libroRepository;

    @InjectMocks
    private LibroConsultaServiceImpl libroConsultaService;

    @Test
    void existeLibro_DebeDevolverTrue_CuandoExiste() {
        when(libroRepository.existsById(1L)).thenReturn(true);
        assertTrue(libroConsultaService.existeLibro(1L));
    }
}
```

`when(...).thenReturn(true)` "programa" el doble de prueba: dice exactamente qué debe devolver, sin ninguna base de datos real respondiendo por detrás.

Sustituibilidad, hecha concreta

El escenario del borrado lógico es la ventaja de sustituibilidad, hecha concreta: el módulo de notas de lectura nunca se entera de cómo cambia el catálogo por dentro, solo sigue llamando a la misma interfaz. Esto es lo que de verdad cambia el día que llegue ese borrado lógico:

	Ficheros a tocar
Con el componente (LibroConsultaService)	Solo LibroConsultaServiceImpl
Sin el componente (cada módulo inyectando LibroRepository)	Cada módulo que lo inyecte — NotaLecturaService, y cualquier otro

Vas a construir exactamente este mismo patrón sobre tu propio proyecto en la Actividad 4.1, con Videojuego/Review en lugar de Libro/NotaLectura.

El mismo patrón, en la otra dirección

Hasta ahora, quien pregunta es `notaLectura` y quien responde es el catálogo (`LibroConsultaService`). El problema también se plantea al revés: `LibroResponseDTO` quiere mostrar la puntuación media de las notas de lectura de cada libro — pero esa información vive en MongoDB, no en PostgreSQL, y `LibroService` no debería tener que saber que Mongo existe siquiera.

Un cambio concreto, del mismo tipo — Mañana decides que las notas marcadas como spam (moderado) no cuenten en la media. Con el componente de por medio, ese cambio se queda contenido dentro de `NotaLecturaConsultaServiceImpl`: `LibroService` sigue llamando a `puntuacionMediaDe(libroId)` exactamente igual, sin enterarse de nada.

Exactamente el mismo molde

Se resuelve con exactamente el mismo molde, solo que ahora la interfaz vive en el módulo de notas de lectura:

```
public interface NotaLecturaConsultaService {  
    double puntuacionMediaDe(Long libroId);  
}
```

```
@Service  
@RequiredArgsConstructor  
class NotaLecturaConsultaServiceImpl implements NotaLecturaConsultaService {  
    private final NotaLecturaRepository notaLecturaRepository;  
    @Override  
    public double puntuacionMediaDe(Long libroId) {  
        return notaLecturaRepository.findByLibroId(libroId).stream()  
            .mapToInt(NotaLectura::getPuntuacion)  
            .average()  
            .orElse(0.0);  
    }  
}
```

Ningún módulo es "el importante"

LibroService inyecta NotaLecturaConsultaService —la interfaz, nunca NotaLecturaRepository ni la clase que la implementa— para rellenar ese campo nuevo de LibroResponseDTO. Ningún módulo es "el importante" al que los demás preguntan: cualquiera de los dos puede exponer un contrato hacia el otro, según quién necesite qué dato — y da igual qué motor de persistencia haya detrás de cada uno, el molde es el mismo.

Se prueba en aislamiento exactamente igual: @Mock sobre NotaLecturaRepository, @InjectMocks sobre NotaLecturaConsultaServiceImpl, sin arrancar Spring ni tocar MongoDB. Vas a replicar este componente en la Actividad 4.2, con ReviewsConsultaService.

Ideas clave

- Un componente es una pieza autocontenida y reutilizable, con una responsabilidad y un contrato público que oculta su implementación.
- Origen histórico: los JavaBeans; en Spring Boot, el equivalente son los beans gestionados por inyección de dependencias.
- Ventajas: reutilización, sustituibilidad, pruebas aisladas, división del trabajo. Inconveniente: más interfaces e indirección.
- Spring (@Service/@Repository/@Component + inyección) ES la "herramienta de desarrollo de componentes" aquí — no hay paleta visual.
- Un JpaRepository ya es un componente básico; interfaz + implementación oculta es el siguiente nivel: contrato en api, menor acoplamiento.
- El problema no es inyectar un repositorio — es inyectar el de otro módulo. Tu propio repositorio nunca ha sido el problema.
- Un componente con una única dependencia se presta a un test aislado con Mockito: @Mock + @InjectMocks, sin arrancar Spring.
- El mismo patrón funciona en cualquier dirección y con cualquier motor detrás — ningún módulo es "el importante".